

The golden set

220 messages from the latest 12% of this brand's conversations, labelled for intent, for what the customer wants done, and for whether a human should take it.

How the messages were chosen

Two strata, kept separable in the output because they answer different questions.

150 drawn uniformly from the gold split. This is the only unbiased view of what real traffic looks like, and it alone estimates the intent mix that the prior-weighted numbers reweight to.

70 drawn to top up thin intents. A uniform 150 leaves the rarest intents with roughly eight examples each, and a per-class score on eight examples has an interval wide enough to be useless. Enrichment buys measurable rare classes at the cost of a label mix that no longer matches traffic, which is why the strata never merge.

The enrichment signal is the nearest cluster centroid from the frozen dev-split clustering, mapped to intents through the hand merge recorded in `intents/taxonomy`. Matching messages against the *text* of each intent definition was tried first and was much worse: it put 87 of 220 under one intent whose definition sits near the centre of everything. Embedding models compare messages to messages well and messages to abstract category prose badly.

Which stratum an example came from is **not stored in `data/gold/`**. It was chosen by an embedding model, so it lives in `data/interim/gold_candidates.jsonl` and is joined at load time. A test enforces that no model-derived field appears in the labelled file.

Messages under 15 characters were excluded. Below that there is no intent to recover and a labeller is guessing rather than labelling.

How the labels were made

This is the part to read carefully.

Pass one, blind. Sanyam labelled the first 52 messages by hand with no reference, working from the taxonomy definitions and the routing rules.

Pass two. Those 52 showed drift against the frozen definitions: `service_outage` used for single-device faults when the definition requires a failure affecting many customers at once, and `app_bug` used for stuttering playback when the definition explicitly excludes it. Rather than continue accumulating inconsistency, the remaining labelling was restarted. All 220 were labelled in one pass by Claude (Opus) applying the frozen definitions and the routing spec, and the result was written to `data/interim/reference_labels.jsonl`.

Pass three. Sanyam reviewed all 220 in the labelling tool, with each example pre-filled and every field overridable, and finalised the final dataset.

So the accurate description is **model-proposed, human-reviewed**. Not hand-labelled from scratch, and not machine-labelled without oversight. Any claim stronger than that in either direction would be false.

Agreement between the two passes

Measured on the 52 messages labelled independently by both, before review.

Field	Cohen's kappa	Raw agreement
Intent	0.53 (moderate)	62%
Handling group	0.71 (substantial)	79%
Route	0.18 (slight)	65%
Ask-type	0.17 (slight)	52%

Kappa asks whether two raters agree more than chance alone would explain, which matters here because both raters mostly say the same thing. On routing, 65% raw agreement collapses to 0.18 once chance is subtracted: both raters answer "auto" most of the time, so most of the agreement is coincidence.

Three things follow.

Intent agreement of 0.53 is a ceiling. If two careful raters applying the same written definitions agree on 62% of messages, no classifier can honestly be called better than that against either rater's labels. Any reported accuracy above it is measuring agreement with one particular rater, not correctness.

Handling agreement is much higher than intent agreement, 0.71 against 0.53. Most of the disagreement is between intents that lead to the same next step, such as `playback_error` against `app_bug`. Operationally those confusions cost nothing, which is the argument for reporting same-next-step accuracy alongside macro-F1.

The ask-type figure is not a finding. The definition of `action` was rewritten after the first 52 were labelled, because it was being read as "wants a fix" rather than "needs their account touched". The two passes used different definitions, so that number measures the change, not the raters.

What is in the file

Messages	220
Uniformly drawn	150
Enriched	70
Marked multi-intent	0
Labelled after revealing the brand's reply	0

Route labels come from the same pass that wrote the intents, which means the routing ground truth and the agent's escalation rules share an author. Sanyam's review is what partly offsets it. This is a real limitation of the routing evaluation and the report repeats it there rather than leaving it here.

Intent mix

Counts are over all 220. The prior is over the 150 uniform draws only, and is what real traffic is estimated to look like.

Intent	Count	Prior
playback_error	43	25.3%
content_availability	42	23.3%
other	38	18.0%
app_bug	35	12.0%
ads_complaint	20	8.7%
account_access	14	4.0%
billing_refund	11	2.7%
live_tv_channels	10	4.0%
watchlist_issue	5	0.7%
service_outage	2	1.3%

Two intents are not measurable. `service_outage` has two examples and `watchlist_issue` five. Per-class F1 on those is noise and is reported only to be honest about the fact. Enrichment was supposed to prevent this and failed: the clusters merged into `service_outage` turned out, on careful reading, to be mostly single-device faults rather than outages. That is a finding about the induced taxonomy, not only about the sample.

`other` holds 38 messages, 17% of the set, and it is the most interesting category in the file. Almost all of them are ordinary answerable questions that the taxonomy simply has no home for: whether Hulu works from Argentina, how to use a student discount, what time a show airs, where the Cloud DVR lives in the redesign, how to turn down the intro volume. Clustering induced categories for things that *break* and missed the entire class of *how do I* questions, because subject is what the words carry and clustering can only see subject.

What the customer wants

	Count	Share
Information would satisfy them	141	64%
Their account must be changed	29	13%
No request; they want to be heard	50	23%

Recorded as a second axis after the induced taxonomy turned out to capture what a message is *about* rather than what would resolve it. Nearly two thirds of inbound messages are answerable by telling the customer something, with no account access at all. That is the single most useful fact in this file for deciding what an automated agent should be built to do.

Routing

149 auto, 71 escalate. The escalation triggers are listed in `gold/spec.py`. The expensive one is repeat contact: anyone who says they already tried something or already wrote in goes to a human, which is about one message in eight here.

Reproducing the sample

```
uv run brandbot label
```

Deterministic under `config.SEED`. Needs a local embedding server for the enrichment step, and produces `data/interim/gold_candidates.jsonl` plus the offline labelling page. It does not need an API key.